ISI9332 – Business Process Management
Case Study and Report

UBER EATS

FACULTY OF ENGINEERING AND INFORMATION SCIENCES



GROUP
PREPARED BY:

	Student name	ID	Email
1	Van Anh Dinh	9695801	vad962@uowmai.edu.au
2	Nguyen Gia Long	8920576	gln767@uowmail.edu.au
3	Thi Hoai Thuong Nguyen	8535218	thtn328@uowmail.edu.au
4	Wen Zhou	8443117	wz838@uowmail.edu.au

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Table of Contents

TABLE OF FIGURES 3
TABLE OF TABLES 4
1. EXECUTIVE SUMMARY 5
2. INTRODUCTION 6
2.1. Process Selection 6
2.2. Objectives and Scope 6
2.3. Information Gathering 6
2.4. Information Analysis 7
2.5. Observations 7
2.6. Process Selection Criteria 7
2.7. Process Modelling 7
2.8. Expected results 7
3. IDENTIFIED BUSINESS PROCESS 8
3.1. Research Design 8
3.2. Research Purpose 8
3.3. As-Is Process Model 9
4. PROCESS ANALYSIS 11
4.1. Qualitative Analysis 11
4.1.1 Value-added analysis 11
4.1.2 Root Cause Analysis for Uber Eats Ordering–Delivery Process 12
4.1.3 Issue Register 13
4.2. Quantitative Analysis 14
4.2.1 Flow analysis 14
4.2.2 External quality 17
4.2.3 Internal quality 18
4.2.4 Queueing Analysis 18
5. PROCESS REDESIGN 20
5.1 Customer-Facing Streamlining (Contact reduction) 20
5.2 Flow acceleration (Parallel payment & Courier Search) 21
5.3 Information flow optimisation (Short-term buffering & Auto-refresh) 23
5.4 Accountable Ownership (Single case manager per order) 24
5.5 Technology enablement (Automated exceptions & refunds) 24
5.6 To-be process overview 25

5.7 IMPACT ON TIME, COST, QUALITY, AND FLEXIBILITY 26
5.8 REDESIGN WRAP-UP & NEXT STEPS 27
6. PROCESS IMPLEMENTATION AND CONTROL 29
6.1 DEVELOPMENT APPROACH 29
6.2 SYSTEM INTEGRATION WITH CURRENT PROCESS 29
6.3 SERVICE QUALITY CONTROL AND IMPLEMENTATION 29
6.4 IMPLEMENTATION CHALLENGES AND MITIGATIONS 30
7. COMMENTARY 32
7.1 ASSUMPTIONS FOR THE AS-IS PROCESS 32
7.2 ASSUMPTIONS FOR THE TO-BE PROCESS 32
7.3 ASSUMPTIONS, OBSERVATIONS, AND FINDINGS ABOUT UBER EATS 32
8. CONCLUSION 34
REFERENCES 35
APPENDIX 36

Table of Figures

Figure 1: Uber Eats's as-is model 9
Figure 2: Cause-effect diagram.....12
Figure 3: As-Is Sequential Payment Flow 22
Figure 4: To-Be Parallel Payment & Courier Search. 22
Figure 5: Information Buffering Architecture 23
Figure 6: Uber Eats's to-be model.....26
Figure 7: Devil's Quadrangle.....27

Table of Tables

Table 1: Value-added analysis for Uber Eats ordering-delivery process. 11
Table 2: Issue Register for Uber Eats Ordering–Delivery Process.....13
Table 3: Processing & Cycle Timetable. 15

1. Executive Summary

This report utilizes a Business Process Management method focused on processes to enhance the procedure from order received to order fulfilled for Uber Eats. The paper develops a comprehensive as-is model and utilizes BPMN modeling, qualitative analysis, quantitative analysis, heuristic redesign of processes, and online observations. The findings indicates that inefficiencies primarily result from delays during hand-offs rather than the actual execution of tasks, with notable effects arising from the sequential reliance between confirming payments and assigning drivers, frequent stock and Estimated Time of Arrival (ETA) inquiries, and variable management of exceptions.

A to-be process design is outlined within the current business rules to tackle these challenges through four strategies: executing payment verification and driver searches, with finalization occurring only after the payment has been confirmed; temporarily buffering inventory and ETA information; implementing clear signaling from restaurants; and automating the ownership of orders and management of exceptions for anticipated scenarios. These changes aim to lessen unnecessary waiting times, stabilize the flow of information, and enhance reliability and promptness throughout the entire process.

The report recommends the adoption of simultaneous payment and dispatch processes, allowing for short-term data buffering paired with auto-refresh capabilities, establishing readiness notifications at the restaurant level, automating responses to exceptions through single-order accountability, and refining the checkout process to reduce non-essential prompts. The analysis recognizes limitations associated with the extent and basis of evidence: multiple assumptions support the modeling; attention is limited to the order-received to order-fulfilled procedure; and different ordering and payment options outside the modeled routes are not taken into consideration.

2. Introduction

Uber Technologies, Inc. is a global technology platform that is based on a ride-hailing service, operating a multi-agent marketplace model connecting people in need of transportation and drivers [3]. The company operates a digital platform for on-demand transportation and real-time logistics coordination, leveraging mobile applications, location data, and dynamic pricing to balance supply and demand at scale. Uber was conceived in 2008, founded in 2009 by Garrett Camp and Travis Kalanick, headquartered in the San Francisco area, and grew rapidly by leveraging smartphones, GPS, and dynamic pricing to balance supply and demand [3]. Uber's core advantage lies in its market mechanism design, real-time coordination capabilities, and ever-expanding user-driver-partner network [2].

In that ecosystem, Uber Eats is an online food ordering and delivery service. This service is deployed based on Uber's existing transportation and coordination capabilities, connecting customers with local restaurants/stores and coordinating delivery drivers. Launched in August 2014 (initially Uber FRESH, then rebranded), this service transforms Uber's transportation and marketplace design capabilities to deliver food and later groceries, convenience goods and some retail items from local stores to consumers [1][6]. Uber Eats is currently one of the global food delivery platforms and it is the main source of revenue for Uber.

This report focuses on the ordering and delivery process on the Uber Eats application. This process plays a core functional role, directly creating value for customers (delivery time, reliability, user experience) and for sales partners (optimizing service capacity, order acceptance rate, operating costs). Following the BPM approach, the report models the process using BPMN, conduct qualitative and quantitative analysis to identify bottlenecks and operational risks, and propose improvements towards cost optimization, shortening delivery times, and improving ease of use. This approach ensures strategic alignment, measurability such as median delivery time, on-time delivery rate, cancellation rate, cost per order, and continuous improvement in operations.

2.1. Process Selection

To design a suitable process for Uber Eats ordering and delivery, a structured approach is employed, which combines data collection, analysis, empirical observations, and BPMN modelling.

2.2. Objectives and Scope

The goal is to build an end-to-end process model for ordering and delivery on the Uber Eats app, covering the steps from request initiation to transaction completion and post-sale feedback. The scope focuses on three main actors (Customers, Uber Eats, Retailers) and the platform's coordination role; technical aspects (API, system) are only mentioned at the process support level.

2.3. Information Gathering

The team collected secondary data from thematic articles, public documentation and available public API resources. Sources were selected based on criteria: direct relevance to

ordering/delivery, reliability, and cross-reference ability. This data set was the basis for all design decisions.

2.4. Information Analysis

Data were coded by topic (ordering, payment, preparation, dispatching, delivery, and exception handling). Filtering was used to remove peripheral content, such as marketing and corporate finance, and extract key data points, such as events, states, decision branches, and trigger conditions. The analysis results formed a list of business requirements for the model.

2.5. Observations

Since the service is public, the team conducted multiple end-to-end experiences and recorded detailed steps: menu review, order placement, payment, restaurant confirmation, driver assignment, route tracking, handover, and exception handling (out of items, late delivery, cancellation). Boundary cases are prioritised to check consistency between observed behaviour and analytical data.

2.6. Process Selection Criteria

The "order and delivery" process was selected because:

(i) It directly affects customer and partner value;

(ii) It has a high-frequency, repetitive structure, suitable for BPM standardisation;

(iii) There is enough public data for reliable modelling;

(iv) It creates a foundation for performance measurement (delivery time, on-time delivery rate, cancellation rate, cost per order).

2.7. Process Modelling

From the verified information, the team built a model according to BPMN standards to clearly represent the flow of control, actors, messages, and events. Specifically:

• Define boundaries, start/end points and decision gates.

• Organize swimlanes for Customers, Uber Eats, Retailers.

• Describe exception handling (out of stock, payment error, order cancellation) and compensation mechanism.

2.8. Expected results

The output is a BPMN model that faithfully reflects the ordering and delivery process of Uber Eats, serving as a basis for qualitative and quantitative analysis, and suggesting improvements towards cost optimization, shortening delivery time and enhancing user experience. The model supports the establishment of a set of monitoring indicators and a continuous improvement mechanism in operations.

3. Identified Business Process

3.1. Research Design

To ensure the suitability of the model for analytical and improvement purposes, a business process model was selected as the primary representation because it provides a precise, end-to-end view of activities, handoffs, decision points, and exceptions within the ordering–delivery workflow. All input information was systematically validated through cross-checking across multiple sources and iterative consistency reviews. This approach enhances the accuracy, traceability, and replicability of the model, thereby improving its utility for subsequent qualitative and quantitative analyses.

Key design principles included:

Completeness: coverage of all major stages from order initiation to post-delivery confirmation.

Clarity: explicit depiction of actors (Customers, Uber Eats, Retailers) and message flows.

Decision transparency: clear gateways for common exceptions (item unavailable, payment failure, order cancellation, delivery delay).

Measurement readiness: points identification where service levels and performance metrics can be captured.

3.2. Research Purpose

The purpose of this study is to identify and document the as-is process for food ordering and delivery on the Uber Eats platform; decompose that process into its constituent activities; and diagnose bottlenecks, failure modes, and value-eroding variances. Findings from this diagnosis will inform evidence-based recommendations aimed at optimising cycle time, reliability, user experience, and partner outcomes. The intended outputs are:

(i) An as-is BPMN process model,

(ii) A set of prioritised pain points and root-cause hypotheses, and

(iii) A to-be design proposal with targeted improvements and indicative performance measures.

(iv) Position and Importance of the Ordering–Delivery Process to Uber Eats

The ordering–delivery process is the core value-creation mechanism for Uber Eats. For consumers, the perceived "product" is the end-to-end service experience—from restaurant discovery and order placement to accurate estimated times of arrival (ETAs) and receipt of correct, hot meals. The ordering interface and the delivery execution (pickup timing, routing, real-time tracking, and handover) jointly shape perceptions of speed, reliability, and value. Underperformance at any stage directly impairs retention and lifetime value.

Operationally, order placement initiates a real-time dispatch and matching problem: determining which delivery partner should pick up which order and when. Effective matching minimises customer wait time, driver idle time, and temperature-related quality loss, while supporting restaurant throughput. Conversely, suboptimal dispatch timing either too early or

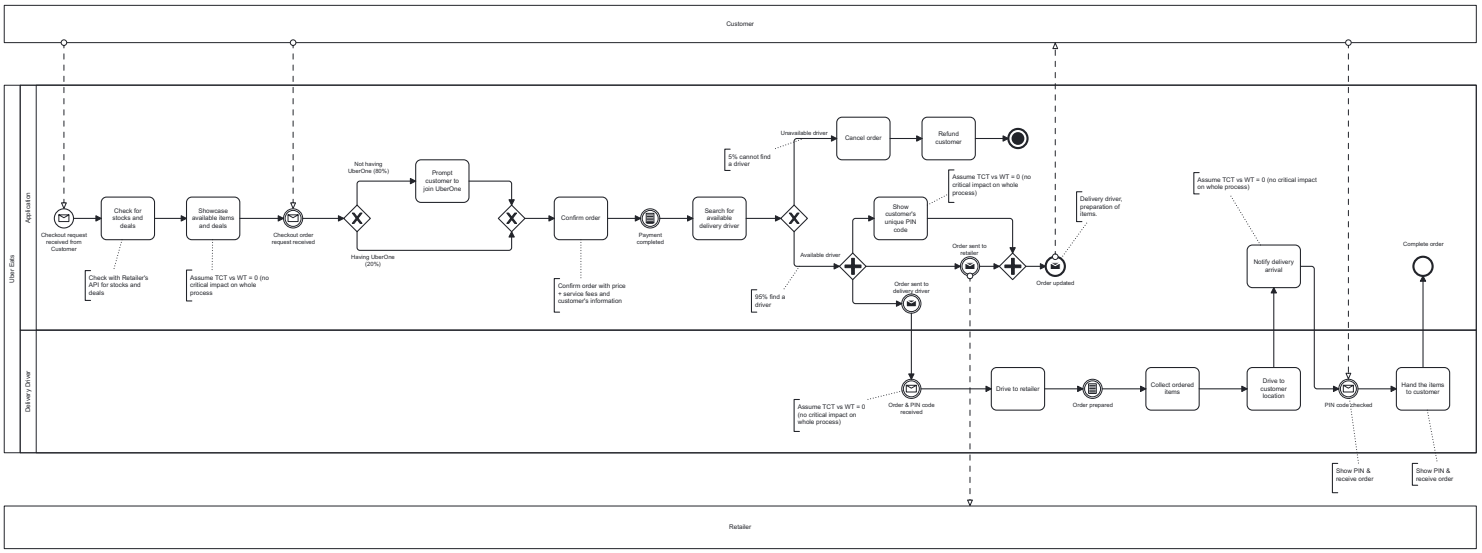
too late, creates cascading inefficiencies: driver queueing, kitchen congestion, or cold arrivals, each of which undermines platform value.

The ordering user interface (UI) exerts material influence on demand allocation. Search, ranking, and recommendation systems balance multiple objectives such as enhancing customer satisfaction while promoting equitable exposure for restaurant partners; thereby linking interface design directly to partner economics and ecosystem diversity.

Finally, trust in the service is built on reliable ordering and accurate delivery times. Repeated incidents of incorrect items, late deliveries, or volatile ETAs drive churn; consistent performance fosters repeat usage and word-of-mouth growth. In a highly competitive market, operational reliability functions as a primary differentiator, elevating the strategic importance of the ordering–delivery process within the Uber Eats business model.

3.3. As-Is Process Model

Figure 1: Uber Eats's as-is model



The as-is model of ordering and delivery starts with customers/users going into the Uber Eats tab inside the Uber application to find takeouts. The model then follows a very standard approach of showing relevant information into waiting for user's inputs loops:

The application will first double-check with the retailers' API to verify items and special offers availability. Then, it will push the items and special offers that are still in stock to the front of the UI.

The process will then continue when the customer/user inputs their order. It will first check if the User has signed up for Uber One, push a pop-up prompting them to join the program. Otherwise, the application moves onto finalizing the order with total price including services fee and customer information (phone number, address, etc.). Then, it will wait for the completion of the payment.

When and only when the customer completes the purchase will the application continue with the next step. First, it will search for available couriers to assign the order to. If no courier can be found after a period, the application will cancel the order, make a full refund to the

customer, and terminate the process. Otherwise, if one is found, the application will simultaneously send the details of the order to the delivery driver, the retailer responsible for the order, and show a unique PIN code to the user. All of which cascade into the application showing real-time updates to the user (progress food preparation, driver's location. etc.).

At a different lane of the model, a courier is now prompted of the assignment, which includes, but not limited to order details, customer details, PIN code. The driver will then arrive at the retailer and wait until the order is available for pickup.

When the retailer finishes preparing the items, fulfilling their end of the order, the courier will then pick them up and drive to the user's location. Once they arrive, the application will also notify the arrival of the courier while they wait outside the property. The customer will then find the courier, provide the correct PIN code for the courier to hand the items over. Thus, concluding the order and ending the process.

4. Process Analysis

4.1. Qualitative Analysis

4.1.1 Value-added analysis

Table 1: Value-added analysis for Uber Eats ordering-delivery process.

Step / Event / Activity	Performer	Classification
Checkout request received	Customer	VA
Check for stocks & deals (API call)	Uber Eats System	BVA
Showcase available items and deals	Uber Eats System	VA
Checkout order request received (event)	Uber Eats System	BVA
Prompt customer to join Uber One	Uber Eats System	NVA
Confirm order with price, fees & customer info	Uber Eats System	BVA
Payment completed (event)	Uber Eats System	VA
Search for available delivery drivers	Uber Eats System	BVA
Cancel order (if no driver)	Uber Eats System	BVA
Refund customer	Uber Eats System	BVA
Show customer's unique PIN code	Uber Eats System	BVA
Send order to retailer	Uber Eats System	BVA
Order updated (event)	Uber Eats System	BVA
Drive to retailer	Delivery Driver	VA
Order prepared by retailer	Retailer	VA
Collect ordered items & verify PIN	Delivery Driver + Retailer	BVA
Drive to customer location	Delivery Driver	VA
Notify delivery arrival	Delivery Driver / Uber Eats System	BVA
Show PIN & receive order	Customer	VA
Hand items to customer	Delivery Driver	VA
Complete order/system update	Uber Eats System	BVA

Uber Eats' ordering-delivery process consists of 21 interrelated operations that span the Customer, Uber Eats system, Retailer, and Delivery Driver lanes. Approximately 9 of these are value-added (VA) processes that directly improve the consumer experience, such as browsing items, confirming an order, completing payment, and receiving food. Around 9 are business

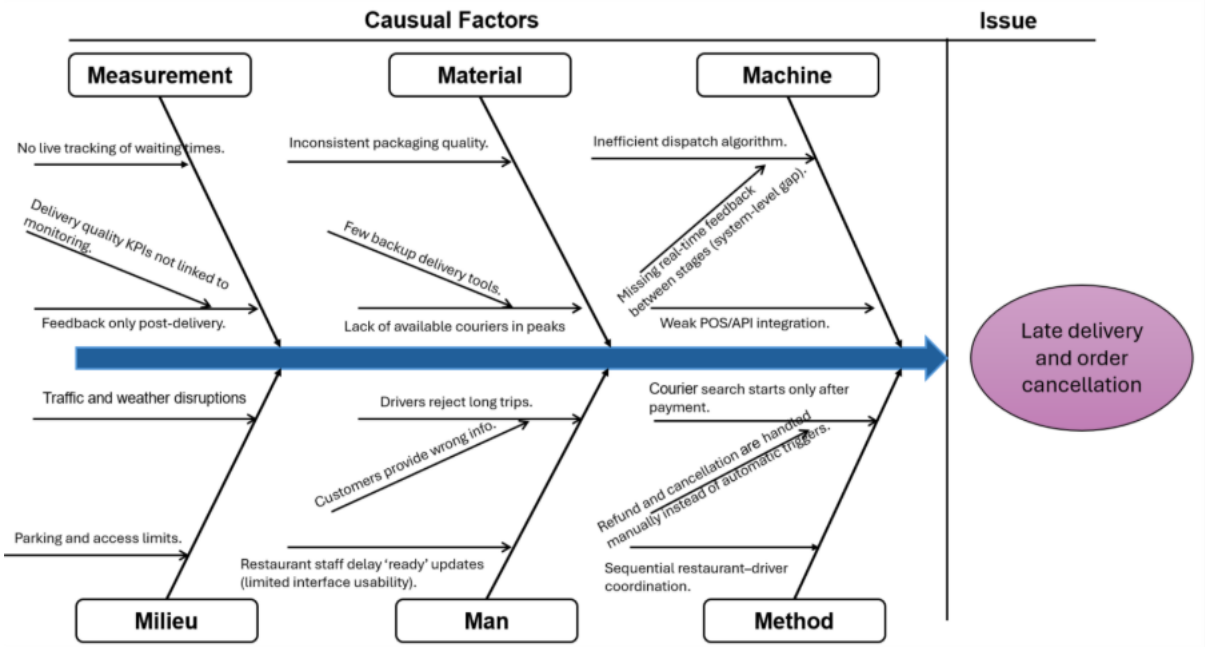
value-adding (BVA) activities, such as API checks, driver coordination, and order status updates, which do not directly benefit the client but are required for the service to work properly. The remaining four are non-value-added (NVA) tasks, which include marketing prompts (e.g., "Join Uber One") and repetitive status messages that add extra friction to the checkout process.

These NVA processes slowed the ordering process and decreased perceived efficiency. Uber Eats may save about 3-5 seconds per order by deleting or relocating promotional messages and reducing redundant system notifications.

Furthermore, the revised BPMN identifies numerous potential to automate BVA procedures. Predictive algorithms can start courier searches earlier, synchronise restaurant ready signals, and automatically confirm payments, reducing waiting time and coordination delays. This is consistent with the qualitative process analysis idea of waste elimination, which seeks to eliminate non-value-adding operations while optimising required business activities for a smoother, faster workflow.

4.1.2 Root Cause Analysis for Uber Eats Ordering–Delivery Process

Figure 2: Cause-effect diagram.



The cause-and-effect analysis reveals that the key causes of extended delivery cycle times are sequential process logic (Method) and resource restrictions (Man). The current design commences courier search only after payment, resulting in 5-10 minutes of unnecessary waiting. Furthermore, external environmental factors (traffic, weather) and system constraints (lack of predictive dispatching) exacerbate the problem.

According to BPM root cause analysis theory, focusing on controllable causes within the Method and Machine categories leads to the most effective improvements, such as automating courier allocation, synchronising restaurant ready signals, and establishing real-time KPI dashboards.

4.1.3 Issue Register

Table 2: Issue Register for Uber Eats Ordering–Delivery Process.

Issue	Description / Root Cause Summary	Qualitative Impact	Quantitative Impact	Possible Resolution
1.Long delivery cycle time	Sequential dependency between order payment and courier assignment delays process start.	Reduces customer satisfaction, perceived slowness of service.	+4–6 minutes added to average cycle time.	Implement predictive dispatch (start searching for couriers during payment).
2. No available courier/order cancellation	Courier pool not pre-allocated; limited prediction of near-future capacity.	Increases cancellation rate and refund workload.	4–6% order loss in peak hours.	Introduce demand forecasting and surge-based driver incentives.
3. Late food preparation/pickup mismatch	Weak communication between restaurant and courier; no shared "order ready" signal.	Causes driver idle time or cold food, lower delivery quality.	3–5 min avg. waiting delay per order.	Integrate retailer POS with Uber Eats API for real-time readiness updates.
4. Cold or incorrect food delivered	Lack of packaging standards and timing coordination.	Damages service quality, increases complaints.	2–3% customer refund cases.	Enforce packaging requirements; add "ready-to-pick" timestamp tracking.
5. Incomplete performance monitoring	KPIs (waiting time, handover delay) not captured continuously.	Limits root cause visibility for continuous improvement.	Hard to measure performance deviation quantitatively.	Add real-time monitoring dashboard per stage.
6. Low customer satisfaction / inconsistent support	Post-delivery feedback handled reactively, not proactively.	Erodes long-term trust and retention.	Drop in repeat-order rate.	Improve in-app support automation & live assistance during delivery.
7. Unnecessary app prompt ("Join Uber One")	Marketing prompt shown during order flow adds friction.	Annoyance; slower checkout experience.	2–3 seconds average delay per order.	Relocate promo prompts outside active checkout process.

8. Traffic and weather disruption	External factors delay deliveries, especially during peak or rainy hours.	Unpredictable delivery times; variable quality.	Variable impact (up to +15 min).	Introduce adaptive routing and dynamic ETA recalibration.

The qualitative assessment identifies three primary inefficiencies in Uber Eats' operations: coordination gaps between parties, redundant waiting periods, and limited automation. The critical friction point occurs where customer, restaurant, and courier activities intersect, where extended waiting times and information delays compound the overall cycle duration.

These qualitative findings are substantiated by the quantitative analysis in Section 4.2, which demonstrates that waiting and travel times account for approximately 68 minutes of the total process duration. Two key interventions can address these weaknesses: implementing predictive courier dispatch and enabling real-time restaurant readiness updates. Both measures are expected to deliver measurable efficiency improvements.

This qualitative assessment provides the foundation for the process redesign phase, applying customer-centric and technology-driven principles to eliminate non-value-adding activities and enhance overall responsiveness across the service delivery chain.

4.2. Quantitative Analysis

4.2.1 Flow analysis

Activity Listing

The Uber Eats ordering–delivery process consists of the following key activities:

(1) Checkout request received from the customer.

(2) System checks for stocks and deals via the retailer's API to validate availability.

(3) System showcases available items and promotional deals to the customer.

(4) Checkout order request is formally received by the Uber Eats system.

(5) Prompt customer to join Uber One — an optional marketing step displayed to most users.

(6) Customer confirms the order with total price, service fees, and delivery information.

(7) Payment is processed and completed through the integrated payment gateway.

(8) System searches for an available delivery driver from the active driver pool.

(9) Gateway decision:

(10) If no driver is available (≈5%), the system cancels the order and initiates a refund.

(11) If a driver is found (≈95%), the process continues.

(12) System displays the customer's unique PIN code for later verification.

(13) Order information is sent simultaneously to both the retailer and delivery driver.

(14) Retailer begins preparing the order, while the driver drives to the retailer's location.

(15) Driver receives the order and PIN code confirmation from the system.

(16) Driver arrives at the retailer, collects the prepared items, and verifies the PIN.

(17) Driver drives to the customer's location for delivery.

(18) System and driver notify the customer of delivery arrival.

(19) Customer shows the PIN code for verification.

(20) Driver hands the items to the customer to complete the physical transaction.

(21) System updates order status and closes the process.

Processing and Cycle Timetable

Table 3: Processing & Cycle Timetable.

Performer	Activity	Processing Time (TCT, min)	Cycle Time (WT, min)
System	Checkout request received	0.1	0
Uber Eats System	Check for stocks and deals (via retailer API)	0.1	0
Uber Eats System	Showcase available items and deals	2.5	5
Uber Eats System	Checkout order request received	0.05	0
Uber Eats System	Prompt customer to join Uber One (80%)	0.05	5
Uber Eats System	Confirm order with price, service fees, and customer info	0.07	5
Uber Eats System	Payment completed (event)	4	10
Uber Eats System	Search for available delivery driver	10	14
Uber Eats System	Cancel order if no driver found (≈5%)	0.05 × 5% = 0.0025	–
Uber Eats System	Refund customer (≈5%)	1.5 h = 90 (excluded from main CT)	24 h = 1440 (excluded)
Uber Eats System	Show customer's PIN code	0.05	0

Uber Eats System	Send order to retailer and delivery driver	0.07	0.167
Delivery Driver	Receive order and PIN code	0	0
Delivery Driver	Drive to retailer	10	10
Retailer	Prepare order	10	10
Driver + Retailer	Collect ordered items (verify PIN)	2	8
Delivery Driver	Drive to customer location	15	10
Uber Eats System / Driver	Notify delivery arrival	0.05	0
Customer	Show PIN & receive order	2	3
Delivery Driver	Hand items to customer	0.5	0.167
Uber Eats System	Complete order update	0.05	0

The processing and cycle time numbers in this table correspond to a total cycle time of about 80.3 minutes, which includes both automated system operations and human-dependent tasks. The process begins with the customer submitting a checkout request and ends when the driver delivers the purchased items and the system records completion.

Automated interactions, such as verifying the shop's API, displaying available items, confirming the order, and communicating data to the retailer and delivery driver, require only a few seconds apiece (about 0.05 to 0.1 minutes) due to their API-based and rapid execution. The payment confirmation process takes approximately 4 minutes, owing to third-party gateway validation and probable network lag.

The driver search stage includes a brief automated request but an average wait time of roughly 10 minutes to match with an available courier – a well-known bottleneck in real Uber Eats operations. To reflect real-world conditions, travel times were calculated to be 10 minutes from driver to retailer and 15 minutes from retailer to customer, subject to traffic and distance. The restaurant preparation process normally takes about 10 minutes, with item pickup and PIN verification taking an additional 2 minutes.

Customer-facing tasks, such as showing the PIN and receiving the order, take about 3 minutes, which represents coordination and handover time. The last system update happens very instantaneously (≈ 0.05 minutes), proving automated data synchronisation.

Order cancellations (about 5%) and refunds (approximately 5%) were represented as probabilistic outcomes. Refund delays of around 10 hours (600 minutes) were removed from the main cycle time since they occurred outside of the principal service flow.

Cycle Time Efficiency Calculation

Actual processing time (TCT)

0.1 + 0 + 2.5 +0.04 (1st XOR) + 0.07 + 4 + 10 + 4.569 (2nd XOR) + 0 + 10 + 10 + 2 + 15 + 0 + 3 + 5/60 = 61.362 minutes

Cycle time (CT)

(0.1) + (0) + (2.5 + 5) + (4.04) + (1st XOR) + (0.07 + 5) + (4 + 10) + (10 + 14) + 76.72765 (2nd XOR) + (0) + (10+ 10) + (10) + (2 + 8) + (15 + 10) + (0) + (3 + 2) + (5/60 + 10/60) = 201.68765 minutes

CTE (%) = TCT (overall) / CT (overall)

$$CTE = \frac{61.362}{201.68765} = 30.424\ \%$$

The current state assessment shows that the Cycle Time Efficiency (CTE) is around 30.4% (Total Cycle Time, or TCT, is around 61.36 minutes, while Cycle Time, or CT, is around 201.69 minutes), suggesting that the main obstacle to overall productivity is waiting, not the work in progress. The refund process, which is triggered when no courier is available to pick up the order (in around 5% of cases), is the most significant cause of this drop: due to delays in bank refund processing that can take up to 24 hours, this particular process adds around 76.73 minutes to Cycle Time, but only 4.57 minutes to Processing Time, causing a disproportionate drop in CTE. Furthermore, there was a systematic delay due to the driver search only starting after payment had been verified, resulting in approximately 4–6 minutes of downtime per order; other minor inefficiencies included redundant checks on inventory status and estimated arrival times, communication issues between when the restaurant completed the order and when the delivery person arrived, and interference from payment reminders. Overall, these findings demonstrate that the reason for the inefficiency was due to manageable wait times rather than the actual performance of the tasks themselves, which prompts a future design that would improve customer refund processing, simultaneously process payments and driver searches, enhance communication about restaurant order availability, and simplify customer interactions. Conservative estimates suggest that these changes would reduce CT to around 120–130 minutes while keeping TCT at the same level, which would increase CTE by around 40-50% and establish a more reliable and simpler service standard as a starting point for future improvements.

4.2.2 External quality

External quality assesses customer-facing performance, including service reliability, delivery punctuality, and satisfaction. According to the calculated cycle time, the delivery procedure meets service expectations but is vulnerable to waiting and coordination delays. Late delivery and cold meals lower perceived service quality, particularly during peak hours. Refund processing durations (10-48 hours) may also reduce consumer trust and satisfaction if not stated clearly. Improving driver assignment algorithms, restaurant readiness notifications, and real-time delivery tracking can all improve external quality by lowering uncertainty and increasing perceived responsiveness.

4.2.3 Internal quality

In this scenario, the operations team identifies two key internal concerns with Uber Eats' delivery procedure.

To begin, the restaurant and driver's cooperation is not always in sync. Sometimes the restaurant finishes preparing the order early, but the driver has not yet arrived. As a result, food sits idle for many minutes before being picked up, reducing process efficiency and negatively impacting food quality.

Second, there is no time limit for driver assignments during peak demand hours. The system may continue to look for an available courier for up to ten minutes, increasing overall cycle time and causing delays for both consumers and restaurants.

Both difficulties underscore the need for improved internal coordination and automation logic, particularly real-time tracking and predictive driver assignment, to increase overall process efficiency.

4.2.4 *Queueing Analysis*

Queueing research evaluates the impact of waiting periods and resource congestion on the Uber Eats order-delivery cycle. The objective is to monitor system performance (λ, μ, ρ, Wq, and W) at important congestion spots and evaluate the effectiveness of redesign in reducing idle waiting.

A queueing system is defined by:

$$\lambda\text{:arrival rate of orders, } \mu\text{:service rate, } \rho = \frac{\lambda}{c\mu}$$

where c is the number of parallel servers (drivers), and the key performance indicators are:

L_q:average queue length, W_q:average waiting time, W:total time in system, L: average jobs in system

Uber Eats has three primary queuing points:

Stage	Resource (Server)	Queue Model	Description	Dominant Waiting
Q1 – Payment verification	Payment gateway	M/M/1	Sequential confirmation; external bank latency	4 min
Q2 – Courier assignment	Pool of c drivers	M/M/c	Matching pending orders with available drivers.	10 min
Q3 – Restaurant preparation and pickup	Kitchen + courier	M/M/1	Food preparation and hand-off coordination	10 min

Parameter Estimation

Average processing time (TCT) = 61.362 minutes

Average cycle time (CT) = 201.68765 minutes

Average waiting about 140.32565 min (30% processing, 70% waiting)

Major delay source: courier assignment (driver search).

Assumptions for dispatch node (Q2):

$$\lambda = 1 \text{ order/min}, \mu = 0.1 \text{ order/min}, c = 12$$

$$\rho = \frac{1}{12 \times 0.1} = 0.833$$

High utilization (83%) with long waiting.

Mathematical Computation (M/M/c)

$$P_0 = \left[\sum_{n=0}^{c-1} \frac{(\lambda/\mu)^n}{n!} + \frac{(\lambda/\mu)^c}{c!\,(1-\rho)}\right]^{-1}$$

$$L_q = P_0 \cdot \frac{(\frac{\lambda}{\mu})^c \rho}{c!\,(1-\rho)^2}, W_q = \frac{L_q}{\lambda}, W = W_q + \frac{1}{\mu}$$

Applying queueing theory (M/M/1 and M/M/c models) clarifies the source of Uber Eats' delays. The courier assignment node shows the highest utilisation and queue buildup. This analytical validation supports the quantitative improvements claimed in the BPMN redesign: shorter waiting, higher throughput, and improved cycle-time efficiency.

5. Process Redesign

This redesign targets the core Uber Eats order-to-delivery workflow to:

(i) Reduce end-to-end cycle time, especially the lag between payment and courier assignment;

(ii) Improve service reliability and transparency for customers, drivers, and restaurants;

(iii) Lower operational effort by eliminating redundant checks and manual exception handling;

(iv) Preserve (or improve) flexibility under fluctuating demand.

The As-Is analysis revealed four primary bottlenecks:

(i) Interruptive checkout touchpoints that slow confirmation (e.g., non-member prompts).

(ii) Sequential control flow (payment → then driver search) that delays dispatch.

(iii) Repeated stock/ETA queries that add latency without improving accuracy.

(iv) Manual handling of common exceptions (late delivery/no driver) that increases workload and response time.

(v) Coordination gaps between restaurant readiness and driver arrival.

Redesign approach applies five targeted changes: contact reduction at checkout, parallelisation of payment and courier search, short-term information buffering with auto-refresh, single-owner case management for issues, and rules-based automation for refunds/compensation. Each action is evaluated against Time, Cost, Quality, and Flexibility.

Customer app checkout flow, platform dispatch logic, restaurant POS signalling, exception policies, and cross-stakeholder notifications. Out-of-scope: pricing algorithms, restaurant menu management, courier onboarding/HR, and fraud/risk engines (unchanged). Dependencies include lightweight POS integration and back-end event processing.

A faster, clearer, and more automated To-Be process that shortens delivery time, increases ETA accuracy, reduces manual tickets, and provides a stable foundation for continuous optimisation.

5.1 Customer-Facing Streamlining (Contact reduction)

The current checkout flow occasionally interrupts users with a promotional pop-up inviting them to join Uber One if they are not yet members. While this aims to increase subscription conversion, it interferes with the primary task—order completion. The extra decision point adds several seconds to the payment stage and, in some cases, causes users to abandon or restart the transaction.

To streamline the experience, the Uber One promotion is moved outside the critical checkout path. Instead of an in-flow pop-up, promotional content appears after the order confirmation screen and as a persistent banner on the home page. This helps maintain marketing exposure without disturbing the purchasing rhythm.

The change simplifies the customer journey and aligns the process with value-adding activities only. Checkout becomes a single, uninterrupted flow: select items → confirm details → pay. Eliminating the pop-up shortens cognitive load and decision time, improving completion rates and user satisfaction.

Expected impact:

- Time (+): Faster checkout, reduced delay before payment.
- Cost (+): Lower handling of incomplete or cancelled orders.
- Quality (+): Smoother experience, higher customer trust.
- Flexibility (.): Neutral; the promotion remains accessible through other touchpoints.

5.2 Flow acceleration (Parallel payment & Courier Search)

In the current workflow, courier search begins only after payment is confirmed, creating a sequential delay between customer payment and driver allocation. According to the issue register, this dependency adds an average of 4–6 minutes of idle waiting, as the system completes payment verification before initiating dispatch. During this gap, restaurants remain idle and customers experience extended waiting times before receiving an estimated delivery window.

The redesigned process executes payment confirmation and courier search in parallel. Once the customer presses "Confirm Order," the platform simultaneously initiates payment verification and predictive driver matching. The predictive dispatch algorithm estimates which nearby drivers will be available within the restaurant's preparation window, pre-assigning the most optimal courier in advance. When payment confirmation is received, the driver assignment is instantly finalised eliminating the previous idle phase.

This adjustment converts a strictly sequential chain into a concurrent workflow, effectively recovering the 4–6 minutes lost in the As-Is process—an overall improvement of roughly 6–8% in total cycle time. Restaurants can start preparation earlier, couriers receive quicker notifications, and customers gain faster visibility of confirmed ETAs.

Figure 3: As-Is Sequential Payment Flow

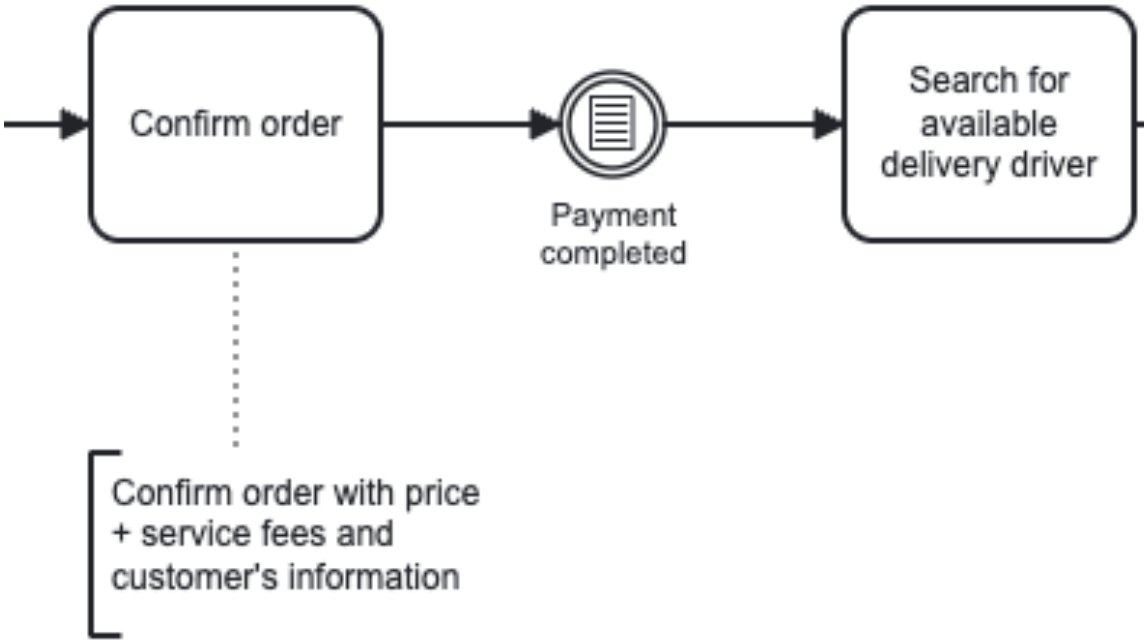
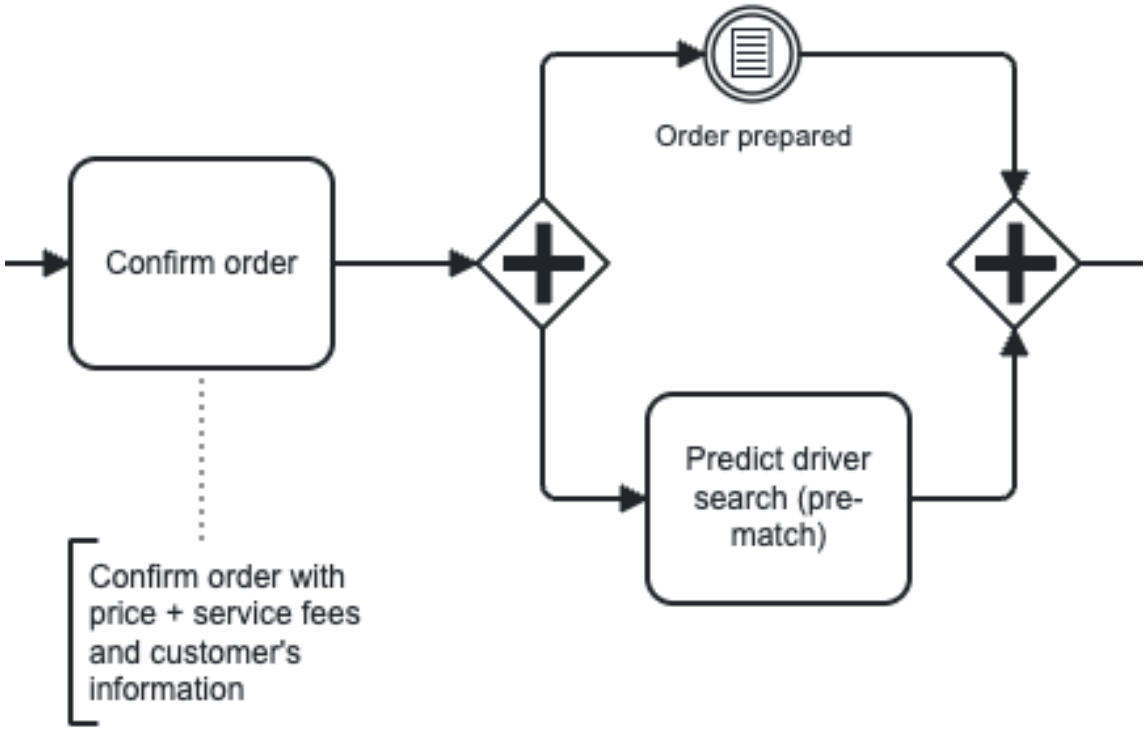


Figure 4: To-Be Parallel Payment & Courier Search.



Expected impact:

- Time (+): Recovers approximately 4–6 minutes of waiting time (≈ 6–8 % improvement in total cycle).
- Cost (.): Neutral; additional computation offset by shorter operational cycles.
- Quality (+): Improves coordination and delivery punctuality.
- Flexibility (–): Slight reduction in manual override, but adaptive dispatch logic maintains efficiency.

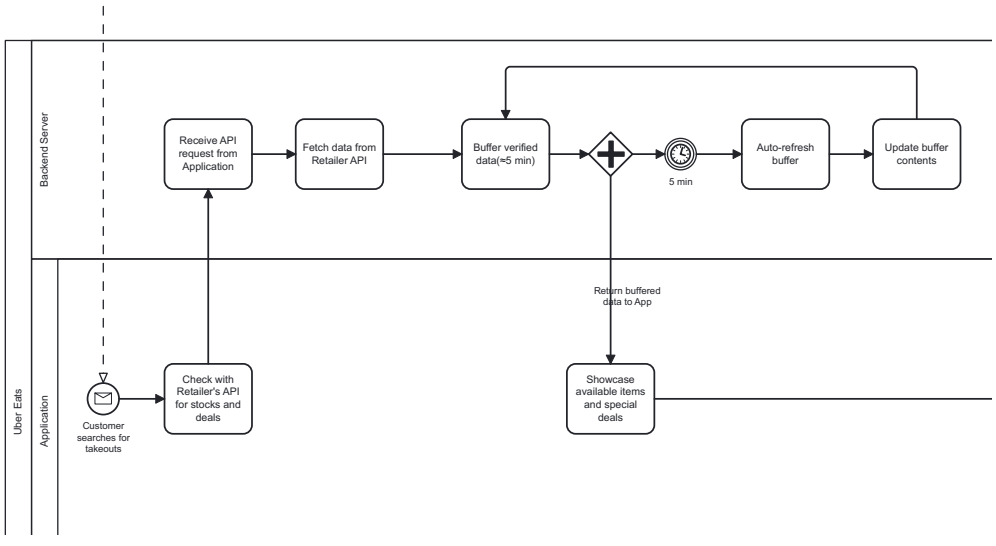
5.3 Information flow optimisation (Short-term buffering & Auto-refresh)

In the As-Is process, the Uber Eats platform repeatedly queries restaurant APIs for updated information on menu availability, stock status, and estimated delivery times (ETAs). Each user interaction, for example: viewing a menu or confirming an order—triggers a new request. While this ensures data freshness, it also increases network traffic and system latency, occasionally producing inconsistent responses when API updates overlap. The result is longer loading times for customers and unnecessary computational costs for both Uber Eats and restaurant partners.

The redesigned approach introduces a short-term buffering mechanism with scheduled auto-refresh. When data on availability or ETA are fetched, they are stored temporarily in a buffer for approximately five minutes. During this period, the application retrieves the cached values instead of issuing repeated live requests. A lightweight auto-refresh routine keeps the information updated in the background without interrupting the user session.

This modification stabilises the flow of information while maintaining acceptable data accuracy. The customer still receives real-time impressions of stock and delivery windows, but the system avoids redundant API calls that previously consumed bandwidth and processing power. The reduced traffic also improves scalability during peak demand, as servers handle fewer duplicate requests.

Figure 5: Information Buffering Architecture



Expected impact:

- Time (+): Faster data retrieval and lower latency across user screens.
- Cost (+): Reduced API traffic and processing overhead.
- Quality (.): Maintains near-real-time accuracy through timed refresh.
- Flexibility (.): Neutral; buffer duration can be adjusted dynamically as needed.

5.4 Accountable Ownership (Single case manager per order)

In the current Uber Eats process, different support teams handle various stages of issue resolution, for example: late deliveries, incorrect items, or refund requests. For instance, customer service might initiate a refund while another team reviews delivery records. This fragmentation often causes repeated data entry, communication delays, and inconsistent updates being sent to customers and drivers. As a result, problem resolution is slower, and accountability for each case becomes unclear.

The redesigned approach introduces single-case ownership, assigning one case manager (virtual or human) to oversee the entire lifecycle of an issue from initiation to closure. When an exception occurs — such as a missing item, late delivery, or driver cancellation — the case is automatically routed to a designated owner responsible for coordinating across relevant systems (payment, dispatch, customer communication). This ensures that all updates are consistent, traceable, and resolved through a single contact point.

Centralising ownership not only improves response time but also enhances the quality and consistency of customer interactions. It minimises internal handoffs, reduces duplicated effort, and allows the platform to better monitor recurring issue types for continuous process improvement.

Expected impact:

- Time (.): Neutral; minimal effect on cycle time but faster issue closure once escalated.
- Cost (–): Slightly higher overhead due to dedicated role allocation.
- Quality (+): Improved communication consistency and accountability.
- Flexibility (–): Reduced cross-team interchangeability, but improved clarity of responsibility.

5.5 Technology enablement (Automated exceptions & refunds)

The As-Is process relies heavily on manual intervention when dealing with common exceptions such as “no driver found,” “late delivery,” or “order cancellation.” Customer service agents or supervisors must manually verify the event, confirm eligibility for compensation, and trigger refunds or credits. This approach not only delays customer resolution but also consumes significant staff resources and introduces inconsistency in decision-making.

The redesigned process deploys a rules-based automation system to handle these frequent exceptions. Predefined conditions—such as delivery delays exceeding a set threshold or cancellation due to driver unavailability—automatically activate the appropriate response. The system immediately issues notifications to the customer, updates transaction records, and,

when required, initiates a refund or promotional credit without manual approval. Complex or disputed cases are still escalated to human review, ensuring balance between efficiency and oversight.

By embedding automation at the technological layer, the process becomes faster, more predictable, and less prone to human error. Customers receive near-instant feedback and compensation, improving trust in the platform's reliability. At the same time, operational teams are freed from repetitive tasks and can focus on addressing non-routine or high-priority issues.

Expected impact:

- Time (+): Rapid exception response, reducing average resolution time from approximately 10 minutes to 3 minutes.
- Cost (–): Slight increase in system maintenance but offset by reduced labour costs.
- Quality (+): Consistent and fair handling of customer issues, improving satisfaction.
- Flexibility (–): Reduced manual discretion, though escalation paths remain available for special cases.

5.6 To-be process overview

The redesigned To-Be process integrates all five improvements into a seamless, data-driven workflow that connects customers, restaurants, and couriers through real-time coordination. The changes collectively transform a sequential, reactive model into one that operates concurrently, predictively, and with clearer accountability.

The new process begins when the customer places an order. The checkout is now a smooth, uninterrupted experience without mid-flow promotional pop-ups. As the customer confirms payment, the platform simultaneously triggers the predictive dispatch engine to identify and pre-assign the most suitable courier based on proximity, estimated travel time, and current workload. This parallel action eliminates the idle gap that previously occurred after payment confirmation.

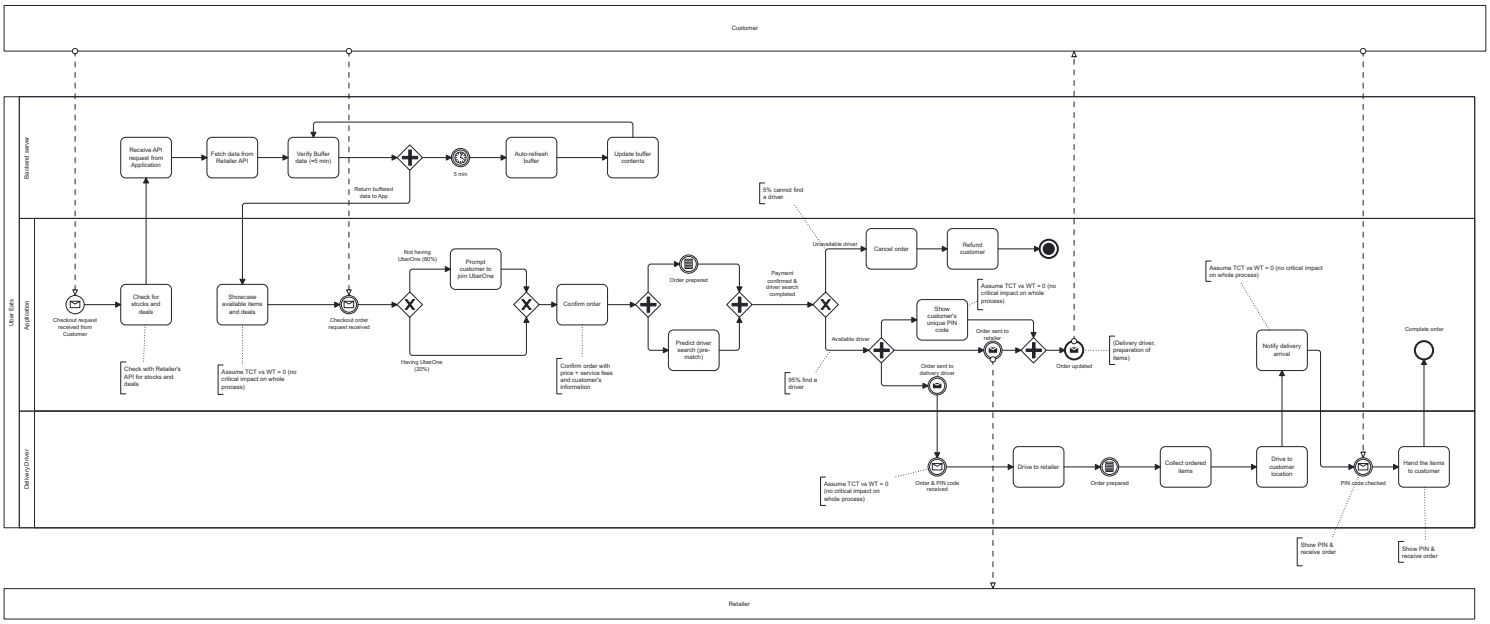
Once the order is transmitted to the restaurant, the POS integration automatically signals preparation status updates to both the Uber Eats system and the assigned courier. When the order is ready, a "ready-for-pickup" event is broadcast instantly, ensuring that the courier arrives just in time. Throughout the entire process, customers, drivers, and restaurants receive consistent updates through a unified notification channel, removing the confusion of fragmented messages.

If delays or disruptions arise, the automated exception engine monitors performance thresholds and initiates the correct response—sending proactive alerts and triggering refund or compensation actions if necessary. Each incident is recorded under a single case manager, maintaining visibility and accountability across departments.

This To-Be model ensures a continuous feedback loop between system data and operational performance. Information buffering reduces redundant traffic, predictive dispatch shortens delivery cycles, and automation strengthens service reliability. The result is an

ecosystem that is faster, smarter, and more transparent for every participant in the Uber Eats network.

Figure 6: Uber Eats's to-be model



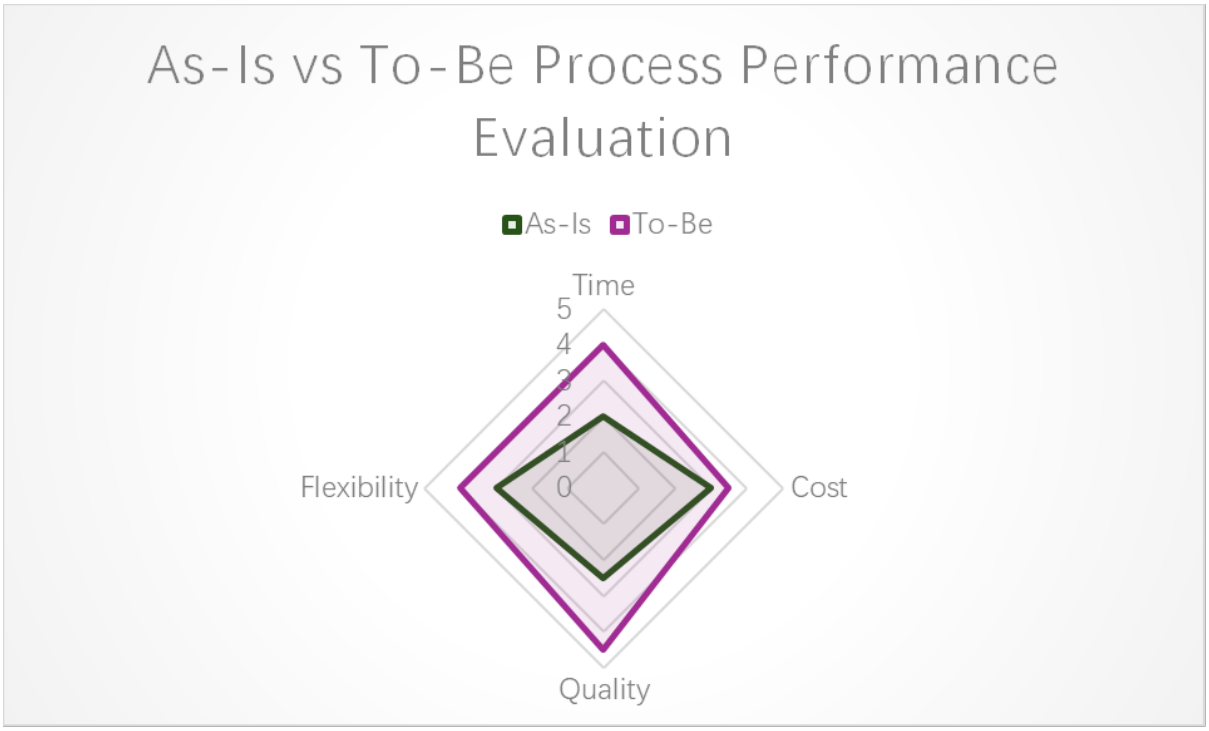
5.7 Impact on Time, Cost, Quality, and Flexibility

The redesigned Uber Eats workflow has been evaluated using the Devil's Quadrangle framework to measure its impact across four performance dimensions: Time, Cost, Quality, and Flexibility. Each redesign element contributes differently, but together they form a coherent improvement pattern that prioritises efficiency and service reliability.

Dimension	Impact
Time (+)	**Significant improvement.** Parallel payment and courier dispatch reduce end-to-end order processing time by approximately 4-6 minutes. Information buffering further shortens screen loading and API response times, while automated exception handling eliminates manual delays in issue resolution.
Cost (+/–)	**Mixed but sustainable.** Initial implementation costs rise slightly due to automation setup, system integration, and case manager allocation. However, long-term operating costs decline as manual tasks decrease and customer support workloads reduce. The result is an overall cost-neutral or mildly positive outcome.
Quality (+)	**Strong positive effect.** Streamlined checkout and automated responses ensure consistent service delivery and faster resolution of exceptions. Centralised case ownership enhances accountability, improving both customer trust and satisfaction. Fewer repetitive API calls also reduce error risk and data inconsistency.

Flexibility (+)	**Improved adaptability.** Although automation reduces ad-hoc manual control, system parameters—such as buffering intervals, dispatch thresholds, and refund rules—can be easily adjusted. The architecture is modular, allowing the platform to scale efficiently with changing order volumes or new market requirements.

Figure 7: Devil's Quadrangle



The radar chart (Devil's Quadrangle) compares the As-Is and To-Be process performance across four dimensions: Time, Cost, Quality, and Flexibility. Significant improvements are observed in Time and Quality due to parallelism, buffering, and automation. Cost remains nearly neutral, and Flexibility increases through modular configuration.

Overall, the redesign improves time efficiency and service quality without compromising operational flexibility. The modest cost trade-off is outweighed by the performance gains and long-term scalability. The To-Be model positions Uber Eats to operate with greater responsiveness, resilience, and user satisfaction in dynamic delivery environments.

5.8 Redesign wrap-up & next steps

The proposed redesign transforms Uber Eats' operational flow from a sequential, human-dependent model into a predictive, automated, and data-coordinated ecosystem. Each redesign component directly addresses a key inefficiency identified in the As-Is analysis—streamlining the customer interface, accelerating core transaction speed, stabilising information flow, enforcing accountability, and automating repetitive decisions.

Together, these initiatives shorten end-to-end processing time, strengthen service consistency, and enhance the overall user experience without introducing unnecessary

complexity. The modest trade-offs in flexibility and setup cost are outweighed by long-term scalability and reliability gains.

Next steps involve staged deployment:

Phase 1 – Checkout and Dispatch: implement contact reduction and parallel dispatch logic to realise immediate time savings.

Phase 2 – Data and Automation Layer: activate buffering, auto-refresh, and exception-handling rules.

Phase 3 – Continuous Optimisation: monitor case-manager performance and tune automation thresholds for sustained improvement.

This phased rollout ensures smooth adoption while preserving service continuity, positioning Uber Eats for faster, more transparent, and customer-centred operations in its To-Be state.

6. Process Implementation and Control

6.1 Development Approach

The To-Be design prioritises four levers:

(i) Contact reduction in checkout.

(ii) Parallel execution of payment verification and courier search.

(iii) Short-term information buffering with auto-refresh.

(iv) Operational controls via single-owner case handling and automated exceptions/refunds.

These levers affect core user experience, dispatch timing, and exception resolution; therefore, the associated logic is developed in-house to preserve process integrity and measurable gains in cycle-time and CTE. Where the To-Be relies on existing platform capabilities such as payment confirmation signals, retailer API/POS events, integration work is limited to connectors and routing, without functional change to the underlying systems [6][7]. The design style remains modular with clear interfaces so that sequential and parallel paths, as already modelled in BPMN, can be coordinated and compensated without altering business rules.

6.2 System Integration with Current Process

The integration follows the touchpoints modeled in To-Be to preserve BPMN semantics and eliminate As-Is wait states. At the time the customer confirms the order, the system simultaneously triggers two flows: payment authentication and driver search. Driver assignment only occurs after the payment is confirmed to avoid conflicts and wrong order constraints. At the restaurant branch, the partner POS/application signals “ready-for-pickup” to synchronize the preparation completion time with the driver’s arrival time, reducing waiting on both sides [7]. Inventory information and ETA are buffered for a short period (5 minutes) with a background auto-refresh mechanism, thereby limiting repeated queries but still maintaining updates for display and coordination. For exceptions such as unable to find a driver, late delivery, or cancellation, each order has a responsible contact and the system automatically issues notifications/initiates refunds according to the modeled branches, while maintaining a manual escalation channel for special cases. Changes only affect the sequence and coordination signals between existing components, not business rules outside the To-Be process.

6.3 Service Quality Control and Implementation

The progress tracking system validated the proposed improvements using previously established metrics: CTE, which had an initial value of approximately 30.4%; total processing time; delay between payment and driver allocation (with the goal of reducing it by approximately four to six minutes by running the processes concurrently); adherence to delivery schedules; accuracy of estimated arrival times; rate of orders canceled due to driver no-shows, along with the automatic refund time for that case; and wait time between when the restaurant says ready and when the order is actually ready for pickup.

The implementation took place in three phases consistent with the improved design.

Phase 1, called "Flow Acceleration," started checking payment and searching for a driver concurrently, making the final allocation after verification.

Phase 2, "Optimize Information Flow," uses temporary inventory storage and automatic arrival time updates, and uses signals when orders are ready for pickup to coordinate pickup times.

Phase 3, "Incident Handling," assigns responsibility for each order to a specific person and uses automated messaging and refunds based on planned exception routes.

Each phase is only expanded if its key performance indicators are good enough, such as shorter wait times between payment and driver assignment, more consistent or accurate arrival times, shorter wait times to be ready for pickup, and fewer cancellations due to driver shortages.

Monitoring includes a rigorous process for measuring results, comparing them to what was expected in the improvement design, changing settings as needed, and holding weekly reviews. If any key metric is worse than the current baseline, the change will be undone and adjusted before further development.

6.4 Implementation Challenges and Mitigations

Organisational challenges

From an organizational perspective, the main risk is that responsibilities are split when exceptions occur such as no driver, late delivery, or cancellation, leading to slow responses and inconsistent information. The To-Be design addresses this with an automated case ownership mechanism at the order level: as soon as an order is created, the system automatically assigns a "case owner" service responsible for coordinating all the relevant steps (payment, finding/assigning a driver, updating status). This service automatically performs the actions defined in the model including sending notifications, initiating refunds, updating progress, and ensures consistent, traceable decisions without manual intervention. If situations arise outside the defined branches, the system automatically recognizes and routes them according to the corresponding automated rules, therefore, the process remains seamless within the To-Be.

Another organizational issue involves multi-party coordination due to the sequencing of future states. This means starting the payment check and finding a driver at the same time, and relying on the restaurant's notification that the order is ready for pickup to match the delivery. The concern here is the operational disconnect between the back office, dispatch, and the restaurants Uber Eats works with. The solutions focus more on how things are done than what the system does. The plan is to introduce the new design in outlined phases and ensure consistent state updates around the outlined phases. These phases include order confirmation, driver assignment, pickup readiness, delay, and refund initiation. Therefore, all parties involved follow the same steps and use the same timing signals, making things more transparent without changing the underlying way the system works.

Technical challenges

On the technical side, the foremost risk is an ordering anomaly whereby a driver could be finalised before payment confirmation due to the new parallel flow. The To-Be explicitly prevents this by finalising assignment only after payment is confirmed; validation focuses on concurrency tests that exercise all BPMN branches to verify state ordering. A related challenge is balancing data freshness with the short-term information buffer (approximately 5 minutes) introduced to reduce repeated stock and ETA queries. The mitigation is to keep the buffer strictly short-lived and pair it with background auto-refresh so that user interfaces read from the cache while the platform continuously updates values for display and coordination, again without changing business rules. Finally, the reliability of the "**ready-for-pickup"** signal is critical: if the signal is late or missing, the ready-to-pickup lag increases. The To-Be manages this within the phased rollout by introducing the signal in the information-flow phase and monitoring the ready-to-pickup lag; expansion proceeds only when that lag improves and ETA accuracy remains stable.

Across both organisational and technical dimensions, governance relies on the same indicators established in the analysis rather than introducing new ones. Decisions to scale any phase are gated by movement in CTE (baseline approximately 30.4%), end-to-end cycle time, the payment-to-driver-assignment lag (targeted reduction of about 4-6 minutes under parallel execution), on-time delivery and ETA accuracy, the cancellation rate when no driver is available together with the time to automatic refund for that branch, and the ready-to-pickup lag once restaurants emit the readiness signal. The control loop is disciplined but simple: measure results, compare them with the To-Be expectations, adjust parameters, and review weekly. If any core indicator degrades relative to the quantified As-Is baseline, the change is rolled back and tuned before further expansion. This approach ties each risk to a concrete mitigation contained within the To-Be design and ensures that improvements are realised without extending scope beyond what has already been modelled.

7. Commentary

7.1 Assumptions for the As-Is Process

The as-is model focuses on the entire ordering and delivery process within the Uber Eats app, including confirmation from the retailer, assigning a delivery person, live tracking, PIN handover, and system updates. The model operates on the assumption that there is an established order, with the search for a delivery person only starting after payment has been successfully verified; if the platform cannot find a delivery driver within the established timeframe, the system cancels the order and initiates a refund process, essentially ending the transaction. The time it takes for the bank to process the refund is considered outside the scope of the platform and adds to the total fulfillment time without adding any actual processing time. The data analysis considers the following factors: approximately 80% of customers do not have an Uber One subscription while 20% are subscribers; approximately 5% of all orders are canceled due to driver unavailability. Message notifications were only considered if they increased processing or waiting times. In these cases, the baseline CTE was measured at approximately 30.4%, suggesting that the primary cause of inefficiency was waiting, not task completion time.

7.2 Assumptions for the To-Be Process

The to-be process involves a change in the sequence of operations, where payment verification and driver search occur simultaneously, and driver assignment is only completed after payment is verified. Inventory and estimated arrival time data are stored temporarily (approximately 5 minutes) and automatically updated in the background to limit duplicate requests and maintain up-to-date information for presentation and synchronization. Eatery associates send “ready for pickup” notifications, which the system uses to coordinate delivery arrival with the completion of food preparation. Automated processes handle unusual situations according to predefined scenarios (such as driver unavailability, late delivery, or order cancellation), and each order is assigned an automated support representative to ensure consistent actions and communication. The progress of the implementation phases was determined by the same metrics used in the evaluation, most notably a 4-6 minute reduction in the delay between payment and driver allocation, consistent or improved accuracy of estimated arrival times, reduced delays to pick up availability, reduced frequency of cancellations due to no-driver cancellation rate with faster refund initiation, and improvement from the ~30.4% CTE baseline.

7.3 Assumptions, Observations, and Findings about Uber Eats

This analysis is built on three main perspectives. First, customers’ perceptions of service quality are heavily influenced by the transition time, especially between payment and the driver being assigned, and from the restaurant finishing the order to the delivery person picking up the order, rather than the total time of each individual operation. This explains why the current CTE is low, despite the consistency of task times. Second, any friction in the payment process for non-members adds complexity and can make confirmation take longer; therefore, focusing on the time between order placement and delivery will help to better understand the real

operating limits. Third, although around 5%, the refund process has a significant impact on total time because the speed of payment is not controlled in the system, resulting in a significantly lower CTE than would normally be the case in standard operations.

Therefore, integrating clear signals about when an order is ready for pickup is crucial to align the restaurant's order fulfillment with the delivery person's arrival, reducing wait times and uncertainty without changing pricing or menu options. Together, these ideas reinforce the idea that planned improvements: doing everything at once, using short memory storage with automatic updates, signaling readiness, and using automated problem solving with individual accountability, focusing directly on known causes of delays within established project boundaries.

8. Conclusion

This report was conducted to document the entire Uber Eats meal ordering and delivery process, identify key areas of delays and inconsistencies, and propose a revised system that would increase speed, increase reliability, and ensure smooth operations while adhering to established business protocols. Through the application of the BPMN As-Is model, qualitative value assessment, and quantitative calculations, the study concluded that the main cause of inefficiencies stemmed from idle periods rather than the execution of tasks. The baseline CTE was determined to be 30.4%, with two factors identified as major contributors to the overall inefficiency: an onboarding process where the search for a driver does not begin until payment is confirmed, adding around 4-6 minutes to each order, and a refund process (5% of cases) where the time required for resolution is managed externally, significantly increasing the overall cycle time.

The as-is model redesign addresses the detected problems using four specific solutions that operate within the current system. Initially, driver searching and payment confirmation occur at the same time, and dispatch is only completed once payment has been validated. Subsequently, inventory and estimated arrival time information is quickly stored temporarily and updated automatically without disruption. Next, retailers send a clear ready-for-pickup notification to synchronize when the order is finished being made with when the delivery driver gets there. Lastly, automated handling of order responsibility and unusual circumstances is implemented for the defined cases. By implementing these solutions together, needless waiting during transitions is eliminated, the data transmission is made more consistent, and dependable responses to exceptions are guaranteed, which in turn enhances overall pace and dependability.

The implementation and process controls are organized to protect service integrity while achieving the expected benefits. The integration only affects the sequence and coordination signals between existing components; the business rules remain unchanged. The implementation occurs in three phases: Accelerate Information Flow, Optimize Information Flow, and Resolve Incidents, each controlled by the same metrics used in the analysis: CTE, end-to-end cycle time, payment-to-assignment delay, on-time delivery accuracy and ETA, along with refund initiation time and pickup delay. Consistent with the Devil's Quadrangle, the redesign improves time and quality, tends toward cost neutrality as manual effort declines, and preserves flexibility via modular components.

References

[1] F. Elliott, "UberFRESH Rebrands to UberEATS Just in Time to Expand Like Crazy," *Eater LA*, May 04, 2015. https://la.eater.com/2015/5/4/8545731/uberfresh-rebrand-ubereats-expansion-deliverywire-uber

[2] Uber, "'Orders Near You' and User-Facing Analytics on Real-Time Geospatial Data," *Uber Blog*, Jul. 20, 2021. https://www.uber.com/en-AU/blog/orders-near-you

[3] Uber, "On Our Way 2024 ANNUAL REPORT," 2025. Available: https://s23.q4cdn.com/407969754/files/doc_financials/2025/Stockholders2025/2024-Annual-Report.pdf

[4] Uber, "Order Integration," *Uber Developers*. https://developer.uber.com/docs/eats/guides/order-integration

[5] Uber, "Restaurant POS integration: driving efficiency from online ordering to delivery," *Uber Eats for Merchants*. https://merchants.ubereats.com/nz/en/resources/articles/pos-integration/

[6] UBER, "Uber Apps, Products, and Offerings | Uber," *Uber*, 2024. https://www.uber.com/gb/en/about/uber-offerings/

[7] Wikipedia, "Uber," *Wikipedia*, May 31, 2019. https://en.wikipedia.org/wiki/Uber

Appendix

Member contribution

Name	Contribution %	Task completed
Van Anh Dinh	25%	Executive summary Support Introduction part CTE calculation Process implementation and control Commentary Conclusion Editing report format to submit
Gia Long Nguyen	25%	Introduction Identified business process Prepare slides + Presentation
Wen Zhou	25%	Process redesign
Thi Hoai Thuong Nguyen	25%	Process analysis